Ders 14 Genişletilmiş Notu: Süreçler Arası İletişim: Paylaşımlı Bellek, Mesaj Geçirme, POSIX, Mach, Soketler, RPC ve Borular

Ders 14 Genişletilmiş Notu: Süreçler Arası İletişim: Paylaşımlı Bellek, Mesaj Geçirme, POSIX, Mach, Soketler, RPC ve Borular

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Dersin Sorusu: fork Sonrası Farklı Olan İki Bilgi
- 2. Süreçler Arası İletişimin İki Modeli
- 3. Bağımsız ve İşbirlikçi Süreçler
- 4. Üretici-Tüketici Problemi ve Sınırlı Tampon
- 5. Paylaşımlı Bellekte Sorumluluk Kimde?
- 6. Mesaj Geçirme Sisteminin Tasarım Soruları
- 7. Doğrudan İletişim
- 8. Dolaylı İletişim ve Posta Kutuları
- 9. Engelleyen ve Engellemeyen İletişim
- 10. Tamponlama
- 11. POSIX Paylaşımlı Bellek
- 12. Mach'ta Mesaj Geçirme
- 13. Windows ALPC
- 14. Soketler
- 15. Uzak Yordam Çağrısı (RPC)
- 16. Borular

1. Önceki Dersin Sorusu: fork Sonrası Farklı Olan İki Bilgi

Ders, çevrim içi yapıldı. Hocanın kamerası çalışmadığı için yalnız ekran paylaşımı ve ses vardı. Diğer öğrencilerin katılmasını beklerken hoca, önceki derste açık bıraktığı soruyu cevapladı.

`fork` çağrısından sonra ebeveyn (parent) ve çocuk (child) süreçlerin süreç kontrol blokları (PCB, process control block) teknik olarak neredeyse aynıdır. Hoca bunları "identical" diye nitelemişti. Ama iki bilginin mutlaka farklı olması gerekir:

1. **Süreç kimliği (PID, process ID):** Her sürecin ayrı bir kimliği olmalıdır. Bu yüzden iki sürecin PID değerleri birbirinden farklıdır.
2. **Ebeveyn süreç kimliği (parent PID):** Her süreç başka bir süreç tarafından yaratılır. Çocuğun ebeveyni onu yaratan süreçtir. Dolayısıyla ebeveynle çocuğun ebeveyn süreç kimlikleri de farklı olur.

Hoca burada bir sınır çizdi. "Her birinin farklı olması" derken aralarında ebeveyn-çocuk ilişkisi olan süreç çiftini kastediyor, sistemdeki bütün süreçleri değil. Çünkü bir süreç birden fazla çocuk yaratabilir; bu kardeş süreçlerin ebeveyn süreç kimlikleri aynıdır.

2. Süreçler Arası İletişimin İki Modeli

Önceki derste süreçler arası iletişimin (IPC, interprocess communication) gerçekleştirilme yollarına başlanmıştı. Hoca konunun yerine oturması için bu kısmı bir kez daha konuştu. Süreçlerin birbiriyle haberleşmesi için iki temel yöntem vardır: paylaşımlı bellek (shared memory) ve mesaj geçirme (message passing).

2.1 Paylaşımlı Bellek

Paylaşımlı bellekte iki süreç fiziksel olarak aynı bellek bölgesine erişir. Bu yapı çoğunlukla dağıtık olmayan, yerel bir sistemde kullanılır. Aynı bilgisayardaki süreçlerin haberleşmesi için idealdir.

Dağıtık sistemlerde de paylaşımlı bellek mümkündür, ama istisnai bir durumdur. Örneğin InfiniBand gibi bir bağlantı üzerinden dağıtık bir paylaşımlı bellek gerçeklenebilir. Sorun, böyle bir bağlantının maliyetli olması ve kolay gerçeklenememesidir. Bu yüzden dağıtık yapılarda genellikle mesaj geçirme kullanılır.

Normalde işletim sistemi, A süreciyle B sürecinin adres uzaylarını (address space) birbirinden yalıtmaya çalışır. A'nın B'nin adres uzayına, B'nin de A'nınkine erişmesini istemeyiz. Paylaşımlı bellek bu kuralın istisnasıdır. Bu nasıl oluyor? İki süreç de karşılıklı olarak bu işe onay verir. Güzel tarafı şudur: gönderilecek mesaj bir kez yazılır, okuyacak süreç de doğrudan oradan okur. Arada kopyalama yoktur.

2.2 Mesaj Geçirme

Mesaj geçirmede ortak bir bellek bölgesi yerine çoğunlukla çekirdek (kernel) üzerinde bir mesaj kutusu ya da port yaratılır. İsimlendirme sistemden sisteme değişebilir. Mesajlar buraya yerleştirilir, diğer süreç ya da süreçler tarafından buradan alınır.

Hoca bu yapının "uzun soluklu" bir iş olduğuna dikkat çekti. A süreci bir mesaj oluşturup mesaj kuyruğuna (message queue) koyuyor, B süreci de oradan alıyor. Mesaj kuyruğu çekirdek seviyesinde gerçeklenmişse, yani araya çekirdek giriyorsa, her işlem için bir sistem çağrısı (system call) yapmak gerekir. Bunun dışında bir de kopyalama yükü vardır:

- Mesaj önce A sürecindedir. A'dan çekirdekteki alana kopyalanır.
- B okuyacağı zaman mesaj kuyruktan B'ye kopyalanır.

Yani aynı bilgi standart yapıda en az iki kez kopyalanır. Bunu atlatmanın yolları vardır; ilerideki örneklerde (Windows ALPC) görülecek. Bu yüzden mesaj geçirme özellikle büyük veri bloklarını aktarmak için pek makul değildir ve genelde küçük verilerin aktarımı için kullanılır.

2.3 Modern İşletim Sistemleri

Günümüzde modern işletim sistemleri her iki yöntemi de gerçekler. POSIX standardı ikisini de tanımlamıştır. Dolayısıyla POSIX ile uyumlu bütün işletim sistemleri hem mesaj geçirme tabanlı hem paylaşımlı bellek tabanlı haberleşme yöntemleri sunar.

2.4 İki Modelde de Senkronizasyon Gerekir

İster bir bellek alanına yazın, ister bir kuyruğa, aynı sorular ortaya çıkar: Mesaj yazıldı mı? Yazıldıysa diğer süreç okuyabilir mi? Okunduysa bir sonraki mesaj nereye yazılacak? Kim bu alanın neresine yazdı, kim nereden okumalı? Alanın kapasitesi sıfır, sınırlı ya da sınırsız olarak düşünülebilir. Kısacası iki modelde de bir senkronizasyon ihtiyacı vardır.

3. Bağımsız ve İşbirlikçi Süreçler

Süreçler farklı ölçütlere göre sınıflandırılabilir. Diğer süreçlerle etkileşimlerine bakarak iki tür vardır:

- **Bağımsız süreç (independent process):** Başka süreçlerle etkileşim içinde değildir. Dolayısıyla diğer süreçlerin çalışmasından hiçbir şekilde etkilenmez.

- **İşbirlikçi süreç (cooperating process):** Diğer süreçlerle haberleşir; aralarında bir bağımlılık (dependency) vardır. Birden fazla süreç bir arada çalışıyorsa bir uyum içinde olmalıdır. Uyumu bozacak herhangi bir hareketten hepsi birden etkilenir.

İşbirlikçi süreçleri neden kullanırız? Hoca slayttaki nedenleri saydı:

- **Bilgi paylaşımı.**
- **Hesaplamayı hızlandırma.** Bir işleme hattı (processing pipeline) kurulabilir. Bu konu iş parçacıklarında (threads) daha çok konuşulacak.
- **Modülerlik.** Hocanın örneği: bir süreç kameradan görüntü alır, bir süreç bunun üzerinde ön işleme yapar, bir başka süreç de asıl işi, örneğin yüz tespitini (face detection) yapar. Böylece iş parçalara bölünür.
- **Kullanım kolaylığı (convenience).**

4. Üretici-Tüketici Problemi ve Sınırlı Tampon

Hoca üretici-tüketici (producer-consumer) probleminden daha önce de bahsedildiğini, ders boyunca birkaç kez daha konuşulacağını söyledi. Bilgisayar sistemlerinde, hem işletim sistemlerinde hem bilgisayar ağlarında, birçok iş bu probleme çevrilebilir ya da bu problemle ifade edilebilir.

Örnek olarak istemci-sunucu (client-server) mimarisiyle haberleşen bir sistem düşünün. Sunucu bilgiyi üreten, istemci tüketen taraf olabilir. Tersi de olabilir; neyin sunucusu olduğuna bağlıdır. Üretici ürettiği bilgiyi ortak bir alana koyar. Tüketici bilgiyi oradan alır, işler ve devam eder.

4.1 Sınırsız ve Sınırlı Tampon

Ortak alan (tampon, buffer) iki türlü düşünülebilir:

- **Sınırsız tampon (unbounded buffer):** Teoride sonsuz bir alandır. Bu, üreticinin hiçbir zaman beklememesi demektir; ne üretirse tampona koyabilir. Teoride mümkün olsa da pratikte böyle bir şey yoktur.
- **Sınırlı tampon (bounded buffer):** Tampon örneğin 5 birimse ve 5'i de doluysa, 6. birim yerleştirilemez. Üretici bekler.

Tüketici tarafında ise iki durumda da kural aynıdır: tamponda bilgi yoksa tüketici bekler, bilgi varsa işler.

4.2 Dairesel Tampon Kodu

Slaytta paylaşılan bilgiyi ifade etmek için `item` adlı bir yapı (structure) tanımlanmış ve bu tipte 10 elemanlık bir tampon oluşturulmuştu. `in` ve `out` adlı iki sayaç (indeks) kullanılıyordu. Aşağıdaki kod, hocanın satır satır anlattığı yapıdan kurulmuştur:

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Üretici:

```
item next_produced;
while (true) {
    /* produce an item in next_produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Tüketici:

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in next_consumed */
}
```

Üretici yeni bir veri üretir. `(in + 1) % BUFFER_SIZE` değeri `out`'a eşitse hiçbir şey yapmaz; oradaki boş deyim (null statement) yüzünden döngüde dönüp durur. Eşit değilse ürettiğini tampona yerleştirir ve indeksi bir artırır. Tampon dairesel (circular) olduğu için indeks bir artınca dizinin sonunu aşıyorsa mod işlemiyle başa döner.

Tüketici tarafında da `while (in == out)` kontrolü vardır. Bunun nedenini anlamak için şu soruyu sormak gerekir.

4.3 Tamponun Boş ya da Dolu Olduğu Nasıl Anlaşılır?

Hoca bu soruyu öğrencilere sordu; cevap gelmeyince kendisi açıkladı. İki indeks vardır: `in` bir sonraki elemanın yazılacağı gözü, `out` bir sonraki adımda çekilecek elemanın bulunduğu gözü gösterir.

Başlangıç durumunu düşünün. Hangi göze yazılması lazım? Sıfırıncı göze. Hangi gözden okunması lazım? Yine sıfırıncı gözden. Ama sıfırıncı gözde ilk adımda hiçbir şey yoktur. Demek ki `in` ile `out`'un eşit olduğu, yani yazılacak gözle okunacak gözün aynı olduğu an, tamponda hiçbir bilgi yok demektir. Tüketici bunu kontrol eder.

Tamponun dolu olduğu da `in`'in bir adım sonrasının `out`'a eşit olmasından anlaşılır. Bu yüzden slaytta bir ipucu verilmişti: "Solution is correct, but can only use `BUFFER_SIZE-1` elements". Yani 10 gözlük bir tamponunuz olmasına rağmen en fazla 9 eleman yerleştirebilirsiniz. Hocaya göre bu çözümün daha iyi, daha optimize hâle getirilmesi gerekir.

Bu davranış küçük bir C programıyla denendi: 10 gözlük tampon, üretici kuralıyla doldurulduğunda 9 elemanda doldu; 9 eleman okunduktan sonra `in == out` koşulu yeniden sağlandı.

> **Ek bilgi (derste anlatılmadı):** Tek bir göz boşa gitmesin diye kullanılan yaygın yöntem, tampondaki eleman sayısını tutan ayrı bir sayaç (`counter`) eklemektir. O zaman boşluk `counter == 0`, doluluk `counter == BUFFER_SIZE` ile anlaşılır. Fakat bu sayaç iki süreç tarafından ortak güncellendiği için yarış durumuna (race condition) yol açabilir. Bu sorun büyük olasılıkla senkronizasyon bölümünde ele alınacaktır.

5. Paylaşımlı Bellekte Sorumluluk Kimde?

Hoca paylaşımlı belleğe tekrar dönerek önemli bir noktayı vurguladı. Normalde işletim sistemi süreçlere ait bellek bölgelerini birbirinden yalıtır. Paylaşımlı bellekte ise kullanıcı süreçlerinin haberleşme isteği yüzünden belli bir alan iki ya da daha fazla süreç arasında paylaştırılır.

Bu haberleşmenin nasıl yürüyeceği, düzgün olup olmadığı ve düzgün olması için gereken önlemler tamamen süreçlerin, yani programcının kontrolündedir. Örneğin bir bellek alanına yazmaya başladınız ve ilk 100 baytı yazdınız. Tüketici nereden okuyacak? A süreci bir bilgi yazdıysa B onun üzerine yazacak mı? Bilgiyi nereden itibaren okuyacak? Bunların hepsi programcıya kalmıştır.

> **Hocanın vurgusu:** Paylaşımlı bellekte senkronizasyonu sağlayacak mekanizmalar işletim sistemi tarafından sunulur, ama senkronizasyonun sağlanması programcının sorumluluğudur.

Senkronizasyon konusu bu derste IPC mekanizmaları konuşulurken kısmen ele alınacak. Asıl ayrıntılı konuşulacağı yer, iş parçacıklarından sonraki bölümdür.

6. Mesaj Geçirme Sisteminin Tasarım Soruları

Paylaşımlı belleğin alternatifi mesaj geçirmedir. Burada süreçler aynı adres uzayını kullanamaz. Dağıtık sistemlerde durum zaten budur: birbirinden bağımsız, ayrı sistemler vardır. Hoca dağıtık olmanın illa kilometrelerce uzaklık gerektirmediğini söyledi; yan yana duran iki bilgisayar da bizim için dağıtık bir sistemdir.

Bir mesaj geçirme sisteminin sunması gereken iki temel ilkel işlem (primitive) vardır: `send` ve `receive`. Bunlara mutlaka verilmesi gereken temel parametre, gönderilecek ya da alınacak mesajdır. Ama çoğu zaman başka parametreler de gerekir. Bunu belirleyen şeyler şunlardır: haberleşme doğrudan mı dolaylı mı olacak, senkron mu asenkron mu olacak, tamponlama nasıl olacak?

6.1 Mesaj Boyutu: Sabit mi, Değişken mi?

Hoca bu seçimin artısını eksisini hem işletim sistemi hem de uygulama geliştirici açısından inceledi.

	Sabit boyut	Değişken boyut
İşletim sistemi	Tampon yönetimi çok kolaydır. Mesaj boyutu 1 KB ise ve 10 mesaj tamponlanacaksa 10 KB'lık bir alan açılır, her mesaj adresiyle yerine yerleşir.	Mesaj sayısı üzerinden gidilemez; alanda kaç mesaj olacağı belli değildir. Mesajın nerede başlayıp bittiğini bilmek için önüne boyutunu belirten bir başlık (header) koymak gerekir. Bu ek yüktür.
Geliştirici	Küçük mesajlarda sorun yoktur, belki biraz alan boşa gider. Büyük mesajlarda ise mesajı parçalamak, karşıya aktarmak ve karşıda yeniden birleştirmek gerekir.	Uygulamanın ihtiyacına göre 500 bayt da, 1500 bayt da gönderilebilir. Geliştiricinin eli rahatlar.

Kısacası mesaj boyutu sabitse işletim sisteminin işi kolay, programcının işi zordur. Değişkense tersi geçerlidir.

6.2 Bağlantının Özellikleri

Haberleşecek süreçler arasında bir iletişim bağlantısı (communication link) kurulması gerekir. Bağlantı tasarlanırken şu sorular konuşulur:

- Bağlantı nasıl kurulur? Doğrudan mı, dolaylı mı?
- Bir bağlantı ikiden fazla süreçle ilişkili olabilir mi? Hocanın örneği: beş süreç var, biri mesaj gönderdi. Diğer dördünden hangisi alacak? Hepsi mi, bir kısmı mı, biri mi? Biriyse hangisi?
- Bağlantı üzerinden aktarılan mesajın boyutu sabit mi, değişken mi?
- Bağlantı tek yönlü (unidirectional) mü, çift yönlü (bidirectional) mü? Yani A'dan B'ye mesaj göndermek için kullanılan bağlantı, B'nin A'ya mesaj göndermesine de imkân veriyor mu?

Hoca son soru için dolaylı iletişimden bir örnek verdi. Her süreç için bir posta kutusu oluşturulursa, A'nın B ile haberleşmesi için B'nin posta kutusuna bir mesaj bırakması gerekir. B'nin A ile haberleşmesi için de A'nın posta kutusuna mesaj bırakması gerekir. Bu durumda iki bağlantı da tek yönlüdür.

7. Doğrudan İletişim

Doğrudan iletişimde (direct communication) haberleşecek süreçlerin birbirini açıkça adlandırması beklenir. Az önceki `send` ve `receive` ilkellerine yalnız mesaj değil, hangi sürece gideceği

ya da hangi süreçten alınacağı da bir parametre olarak verilir:

- `send(P, message)`: P sürecine bir mesaj gönder.
- `receive(Q, message)`: Q sürecinden bir mesaj al.

Böyle bir yapıda bağlantının özellikleri şunlardır:

- Bağlantı otomatik olarak kurulur. Q süreci içinden P'ye `send` dendiğinde Q ile P arasında doğrudan bir bağlantı oluşur.
- Bağlantı yalnızca bu iki süreç arasında kurulur.
- Her süreç çifti arasında her zaman tam olarak bir bağlantı vardır.
- Bağlantı tek yönlü olabilir, çift yönlü de olabilir.

Hoca IPC yapılarından örnekler verdi. İlk dönem yapılardan olan boru (pipe) ve FIFO tek yönlüdür. Sonradan POSIX kapsamında sunulan paylaşımlı bellek ve mesaj kuyruklarında ise çift yönlü haberleşmeden bahsedilebilir.

Hoca bir de şuna dikkat çekti: tek yönlü bir yapı, kullanım sırasında her zaman tek yönlü kalmak zorunda değildir. Örneğin bir boru tek yönlüdür. Ama iki boru oluşturup birini bir yönde, diğerini öbür yönde kullanırsanız yapıyı çift yönlü hâle getirirsiniz. Yapının kendisi tek yönlüdür, kullanımı çift yönlü olur.

8. Dolaylı İletişim ve Posta Kutuları

Dolaylı iletişimde (indirect communication) bir süreç mesajı doğrudan başka bir sürece göndermez. Mesajı, haberleşmek istediği sürece ait bir yapıya gönderir. Bu yapıya çoğu zaman posta kutusu (mailbox) ya da port denir. Hocanın benzetmesiyle bu tam olarak gerçek hayattaki posta kutunuza benzer: sizinle haberleşmek isteyen herkes posta kutunuza bir mektup atar, siz de oradan alırsınız.

İlkel işlemler artık posta kutusunu adresler:

- `send(A, message)`: A posta kutusuna mesaj gönder.
- `receive(A, message)`: A posta kutusundan mesaj al.

8.1 Bağlantının Özellikleri

Posta kutusu üzerinden kurulan bağlantının özellikleri doğrudan iletişimden farklıdır:

- Bağlantı yalnızca bir posta kutusunu paylaşan süreçler arasında kurulur.
- En belirgin fark: bağlantı yalnız iki süreç arasında olmak zorunda değildir. Slaytta "link may be associated with many processes" yazıyordu. Nedeni basittir: bir posta kutusu birkaç süreç tarafından kullanılabiliyorsa, o posta kutusuna erişen bütün süreçlerin haberleşebilmesi, yani bağlantının hepsi arasında kurulması gerekir.
- İki süreç birden fazla posta kutusu kullanabilir. Dolayısıyla aralarında birden fazla bağlantı olabilir.
- Bağlantının tek yönlü mü çift yönlü mü olacağı, sistemin gerçeklenmesi sırasında alınacak kararlara bağlıdır.

Dolaylı iletişim için gereken temel işlemler şunlardır:

- Yeni bir posta kutusu (port) yaratmak.
- Posta kutusu üzerinden mesaj göndermek ve almak.
- Posta kutusunu yok etmek. Sisteme göre bu işlem `destroy`, `terminate` ya da `kill` gibi bir ilkelle yapılır.

Hoca simetri farkına da dikkat çekti. Doğrudan iletişimde gönderen karşı tarafın kimliğini verir, alan da karşı tarafın kimliğini verir; süreçler birbirini karşılıklı adresler. Bu simetrik bir yapıdır. Dolaylı iletişimde ise ortak bir posta kutusu vardır ve herkes o posta kutusunu adresler.

8.2 Posta Kutusu Paylaşımı Sorunu

İkiden fazla süreç aynı posta kutusunu paylaşırsa karmaşık durumlar ortaya çıkar. Hocaya göre paylaşan sayısı arttıkça sorun çıkması gerçek hayatta da böyledir.

P1, P2 ve P3 aynı posta kutusunu paylaşsın. P1 posta kutusuna bir mesaj gönderdi. Bu mesajı kim alacak, P2 mi P3 mü? Tasarımcı olarak karar verip uygulayabileceğiniz birkaç yöntem vardır:

- **Paylaşımı sınırlamak:** Bir posta kutusunun en fazla iki süreç tarafından paylaşılmasına izin vermek.
- **Tek alıcı:** Herhangi bir anda yalnız bir sürecin `receive` işlemini çalıştırmasına izin vermek. Hocaya göre üç sürecin bir posta kutusunu paylaşmasına izin verdiğiniz bir durumda bu çok mantıklı değildir ve daha büyük karmaşaya neden olabilir.
- **Sistemin seçmesi:** Mesaj, diğer iki süreçten birine sistem tarafından devredilir. Sistem alıcıyı önceden atanmış bir önceliğe (priority) göre ya da başka bir yöntemle seçebilir. Tek şart, mesajın kime verildiğini gönderene bildirmesidir. Posta kutusunu on süreç kullanıyorsa sistem "bu mesajı şuna verdim" diye geri dönüş yapabilmelidir.

9. Engelleyen ve Engellemeyen İletişim

Mesaj geçirmede gönderme ve alma işlemleri engelleyen (blocking) ya da engellemeyen (non-blocking) yapılabilir. Engelleyen işlem senkron (synchronous), engellemeyen işlem asenkron (asynchronous) olarak da adlandırılır.

- **Engelleyen gönderme (blocking send):** Gönderen, mesaj karşı tarafa tam olarak iletilene kadar bekler. Dolaylı iletişimde A, B'nin posta kutusuna mesaj koymaya gider ama posta kutusu doluysa (örneğin 10, 8 ya da 100 mesaj kapasiteli ve şu an o kadar mesaj varsa) beklemesi gerekir. Doğrudan iletişimde ise karşı taraf mesajı alana kadar bekler.
- **Engellemeyen gönderme (non-blocking send):** Gönderen mesajı gönderir ve yoluna devam eder. Mesajın karşıya gidip gitmediğine, okunup okunmadığına bakmaz. Başka mesajı varsa onu da gönderir.
- **Engelleyen alma (blocking receive):** Alıcı mesaj gelene kadar bekler.
- **Engellemeyen alma (non-blocking receive):** Alıcı mesaj var mı diye bakar. Okuyabiliyorsa okur. Mesaj yoksa ona null döner ve alıcı devam eder.

Hoca engelleyen yapıyı WhatsApp ile anlattı. Bir SMS gönderip hayatınıza devam ettiğinizde engelleyen bir taraf yoktur. Ama WhatsApp'ta mesajı yazıp gönderdiğinizi, önce tek tik, sonra çift tik çıktığını, tikler maviye dönüp mesajın okunduğunu gördükten sonra ikinci mesajı yazdığınızı düşünün. Her seferinde mesajın iletildiğini ve okunduğunu görüp ondan sonra göndermeye devam ediyorsanız bu engelleyen göndermedir. Biri size mesaj atacak diye WhatsApp'ı açıp telefon önünüzde mesaj gelene kadar bekliyor, mesaj gelince okuyup hayatınıza devam ediyorsanız bu da engelleyen almadır.

Engelleyen yapı bu anlamda senkrondur: birinin devam edebilmesi için diğerinin göndermesi ya da okuması gerekir; iki taraf bir noktada buluşur. Engellemeyen yapılar ise asenkrondur; aralarında bir eş zamanlılık yoktur.

9.1 Randevu ve Üretici-Tüketici

Gönderme ve almanın engelleme türleri sistem içinde farklı biçimlerde bir arada bulunabilir. Gönderme engelleyen diye almanın da engelleyen olması gerekmez. Sistem bu yapıları kullanırken size yapılandırma (configure) imkânı da verebilir. Her kombinasyon mümkündür.

Hem gönderme hem alma engelleyen olduğunda iki süreç tam bir randevu (rendezvous) usulüyle çalışır. Bu durumda üretici-tüketici problemi çok basitleşir. Üretici bir eleman ürettiğinde yalnız `send` der. Karşı taraf `receive` edene kadar bekler, sonra dönüp yeniden `send` eder. Tüketici de `receive` ile mesaj gelmesini bekler, alır, devam eder ve yeniden `receive`'de bekler. Bilgi tüketilmeden yenisi üretilmediği için arada bir sorun çıkmaz; ayrıca bir şeyi kontrol etmeye neredeyse gerek kalmaz.

Ama bu yaklaşım belli açılardan verimli değildir. İki süreci birbirine şartlandırmış olursunuz. Farklı hızlarda çalışmalarına izin vermezsiniz.

10. Tamponlama

Bir iletişim bağlantısında mesaj alıp verildiğinde, mesajların geçici olarak tutulduğu bir tampon alanı, yani bağlantıya bağlı bir mesaj kuyruğu beklenir. Bu kuyruğun kapasitesi üç türlü olabilir:

- **Sıfır kapasite (zero capacity):** Kuyruk vardır ama kapasitesi sıfırdır. Elinizdeki bilgiyi koyabileceğiniz geçici bir alan yoktur. Bir mesaj göndermek istiyorsanız karşı tarafla aynı anda buluşmanız ve onun mesajı alması gerekir. Bu durumda başka seçenek yoktur: gönderme engelleyen olmalıdır, yani randevu sistemiyle çalışılır.
- **Sınırlı kapasite (bounded capacity):** Kuyruk örneğin n mesajdan oluşur. Kuyrukta yer varsa gönderme engellemesizdir; mesajı koyup devam edersiniz. Ama 10 gözlük kuyrukta 10 mesaj varken 11. mesajı koymak isterseniz yer açılana kadar beklersiniz. Gönderme işleminiz bir anda engelleyen hâle gelir.
- **Sınırsız kapasite (unbounded capacity):** Gerçek anlamda böyle bir şey yoktur. Olsaydı gönderen hiçbir zaman beklemezdi; her seferinde mesajını koyup devam ederdi.

11. POSIX Paylaşımlı Bellek

IPC sistemlerinin ilk örneği POSIX'in sunduğu paylaşımlı bellek yapısıdır. Hoca önce genel fikri verdi: işlemler dosya işlemlerine benzer, çünkü belleğe eşlenmiş dosyalar (memory-mapped files) kullanılır. Bir dosya oluşturur, boyutunu belirler, sonra bu dosyanın belleğe eşlenmesini sağlarsınız.

> **Ek bilgi (derste anlatılmadı):** Belleğe eşleme (memory mapping), büyük olasılıkla en basit tanımıyla, bir dosyanın ya da paylaşımlı bellek nesnesinin içeriğinin sürecin adres uzayında bir bölgeye bağlanmasıdır. Bu bölgeye yapılan okuma ve yazmalar `read`/`write` sistem çağrısı gerekmeden doğrudan bellek erişimi gibi yapılır. `MAP_SHARED` ile eşlendiğinde, aynı nesneyi eşleyen başka bir süreç yapılan değişiklikleri görür.

Kullanılan fonksiyonlar şunlardır:

1. **`shm_open`:** Paylaşımlı bellek nesnesini açar. Nesneye bir ad verilir; nesnenin mutlaka bir adı olmalıdır.
   - `O_CREAT`: nesne yoksa yaratılmasını sağlar.
   - `O_RDWR`: hem okuma hem yazma yönünde çalışılacağını söyler.
   - `0666`: erişim hakları. Haklar, bu bölgeye ulaşabilecek diğer süreçleri sınırlar; sistemdeki herkesin her türlü erişmesini istemeyebilirsiniz.
   - Dönen değer bir dosya tanımlayıcısıdır (file descriptor). Üst seviyedeki `fopen` değil, alt seviyedeki `open` düşünülmelidir: açılan her dosya için bir tanımlayıcı döner.
2. **`ftruncate`:** Açılan nesneyi belli bir boyuta getirir. Örnekte boyut 4096 bayttır.
3. **`mmap`:** Tanımlayıcıyı belleğe eşler. Dönen `ptr`, artık bu eşlenmiş alanın bellekteki adresidir.
4. **`shm_unlink`:** Nesneye artık ihtiyaç kalmayınca onu kaldırır.

Hoca slayttaki bir alt çizgi sorununa da değindi: slaytta `O_CREAT` ve `O_RDWR` içindeki ve `shm_fd` adındaki alt çizgiler görünmüyordu; doğrusu alt çizgilidir.

11.1 Üretici ve Tüketici Örneği

Slayttaki örnekte boyut 4096, nesnenin adı "OS" olarak verilmiş ve iki mesaj tanımlanmıştı. Üretici iki mesajı art arda yazar. Her yazmadan sonra `ptr`'yi mesajın uzunluğu (`strlen`) kadar ilerletir. Örneğin "Hello" yazıldıktan sonra bir sonraki yazma `ptr`'den "Hello" kadar sonraya yapılmalıdır ki ilk mesajın üzerine yazılmasın.

Tüketici aynı adla nesneyi açar, ama yalnız okuma için (`O_RDONLY`), çünkü tüketicinin buraya bir şey yazmasına gerek yoktur. Bu yüzden boyutu yeniden ayarlamaz. Tanımlayıcıyı belleğe eşler ve `ptr`'den mesajı okur. Üreticinin iki parça hâlinde yazdığı mesaj tek seferde ekrana yazdırılır. Sonra nesne `shm_unlink` ile kaldırılır.

Aşağıdaki kod, hocanın anlattığı adımlardan ve slayttaki adlardan kurulmuştur. Mesaj metinleri "Hello" ve "World!" olarak alınmıştır.

Üretici (`producer.c`):

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main(void)
{
    const int SIZE = 4096;          /* paylaşımlı bellek nesnesinin boyutu */
    const char *name = "OS";        /* nesnenin adı */
    const char *message_0 = "Hello";
    const char *message_1 = "World!";
    int shm_fd;
    char *ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = (char *) mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);
    return 0;
}
```

Tüketici (`consumer.c`):

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main(void)
{
    const int SIZE = 4096;
    const char *name = "OS";
    int shm_fd;
    char *ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = (char *) mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    printf("%s\n", ptr);
    shm_unlink(name);
    return 0;
}
```

İki program `gcc -Wall producer.c -o producer -lrt` biçiminde uyarısız derlendi. Önce üretici, sonra tüketici çalıştırıldığında çıktı `HelloWorld!` oldu.

> **Hocanın vurgusu:** Bu örnek çok mantıklı ya da makul bir uygulama değildir. Yalnızca bu yapıların nasıl kullanıldığını görmeniz için yazılmış basit bir uygulamadır.

Hoca POSIX paylaşımlı bellek ve diğer IPC yöntemlerinin uygulama (laboratuvar) saatlerinde ayrıca gösterileceğini söyledi. Mesaj geçirme tarafında POSIX'in mesaj kuyrukları (message queues) vardır; bunlar da büyük olasılıkla üçüncü uygulamada konuşulacak.

12. Mach'ta Mesaj Geçirme

Mach çekirdeği, daha önce çekirdek yapıları konuşulurken bir mikroçekirdek (microkernel) örneği olarak görülmüştü. Mach'ta haberleşme mesaj geçirmeyle yapılır. Bunun için posta kutusu yapıları kullanılır, ama Mach'ta bunlara port denir. Sistem çağrıları bile mesaj tabanlı olarak gerçekleştirilir.

12.1 Görevler ve Varsayılan Portlar

Mach'ta süreçlere görev (task) denir. Bir görev oluşturulur oluşturulmaz varsayılan olarak iki posta kutusuna sahip olur:

- **Kernel portu:** Adı üstünde, çekirdeğin görevle haberleşmesi içindir.
- **Notify portu:** Olayların (event) çekirdek tarafından göreve bildirilmesi içindir.

Mesaj aktarımı için toplam üç sistem çağrısı gerekir: gönderme (send), alma (receive) ve RPC. RPC bölümün sonunda ayrıca konuşuldu; teknik olarak yerelde yaptığınız bir fonksiyon çağrısını dağıtık bir sistemde de yapabilmektir. Bunlardan başka bir işleme daha ihtiyaç vardır: varsayılan portların dışında yeni portlar oluşturabilmek. Bunun için bir port ayırma (port allocate) ilkeli vardır.

12.2 Mesaj Sırası ve Boyutu

Bir posta kutusu yaratıldığında içinde hiç mesaj yoktur. Mesajlar aktarıldıkça kapasite azalır ve sonunda posta kutusu dolar.

A, B ve C süreçlerinin hepsi D'nin posta kutusuna mesaj göndermeye çalışsın. Mach, A, B ve C'nin gönderim sıraları arasında bir garanti vermez. Ama A'nın gönderdiği bütün mesajların gönderdiği sırayla, yani ilk giren ilk çıkar (FIFO, first in first out) sırasıyla kuyruğa gireceğini garanti eder. Örneğin A bir mesaj, sonra B bir mesaj, sonra A bir mesaj daha, sonra B bir mesaj daha gönderdi. Siz kuyrukta A1, B1, A2, B2 sırasını görmek istersiniz. Mach bunu garanti etmez. Ama A1'in A2'den, B1'in de B2'den önce olacağını garanti eder.

Mach mesajların değişken boyutlu olmasına izin verir. Bunun için mesajın önüne bir başlık alanı ekler ve bu başlıkta mesajın gerçek boyutu belirtilir.

12.3 Posta Kutusu Doluyken Gönderen Ne Yapabilir?

Posta kutusu dolduğunda teknik olarak dört seçenek vardır:

1. **Süresiz beklemek:** Posta kutusunun boşalmasını beklemek. Bu hemen de olabilir, çok uzun sürede de olabilir.
2. **Zaman aşımıyla beklemek:** Beklemeyi belli bir zaman aşımı (timeout) süresiyle ilişkilendirmek. Hoca bu seçeneğin yalnız Mach'ta değil, POSIX'te mesaj kuyrukları gerçeklendiğinde de bulunduğunu söyledi.
3. **Hemen geri dönmek:** Posta kutusunun dolu olduğunu görüp gönderemediğinizi belirterek hemen geri dönmek.
4. **Mesajı geçici olarak önbelleğe almak (cache):** Mesaj işletim sistemine devredilir. İşletim sistemi bizim yerimize kuyruk boşalana kadar bekler. Kuyruk boşalınca mesajı yerine yerleştirir ve gönderene bununla ilgili bir bildirim (notification) gönderir.

Hoca dördüncü seçenekteki sorunu öğrencilerin tahmin etmesini istedi. Herhangi bir posta kutusu, hatta bütün posta kutuları doluyken bir süreç sürekli göndermeye çalışırsa işletim sistemine sürekli önbelleğe alma yükü gelir. Bu uygulanabilir (feasible) değildir. Bu yüzden önbelleğe alınabilecek mesaj sayısı sınırlandırılmıştır. Hoca bu sınırı "mesaj kutusu başına bir" olarak verdi.

> **Ders sonrası not:** Hoca önbelleğe alınabilecek mesaj sayısının posta kutusu başına bir olduğunu söyledi. Ders kitabındaki kural büyük olasılıkla şudur: dolu bir posta kutusu için, belirli bir gönderen iş parçacığının aynı anda yalnız bir bekleyen mesajı olabilir. Yani sınır gönderen başınadır.

12.4 Posta Kutusu Kümeleri

Mach'ta alma işlemini tek bir posta kutusu üzerinden yapmak zorunda değilsiniz. Bir grup posta kutusu birleştirilip bir posta kutusu kümesi (mailbox set, port set) olarak tanımlanabilir. O zaman alma işlemi bu küme içindeki herhangi bir posta kutusundan ya da belirli birinden okuma yapabilir. İki durumda da mesaj yoksa iki seçenek vardır: hiç beklemeden devam etmek ya da belli bir süre beklemek.

13. Windows ALPC

Benzer bir mesaj geçirme yapısı Windows'ta da vardır. Adı gelişmiş yerel yordam çağrısıdır (ALPC, Advanced Local Procedure Call); eski adıyla LPC de denir. Adındaki "local" sözcüğünden de anlaşılacağı gibi aynı makinedeki süreçler arasında çalışır.

Haberleşen süreçlerin bir tarafı istemci, diğer tarafı sunucu olarak adlandırılır:

1. Her sunucu süreç, kendisiyle haberleşmek isteyenlerin bağlanabileceği bir port yayımlar (publish). Buna bağlantı portu (connection port) denir.
2. Sunucuyla haberleşmek isteyen süreçler isteklerini bu porta gönderir.
3. İsteğe bağlı olarak bu iki süreç arasında bir haberleşme yapısı, yani bir ALPC kanalı oluşturulur. İstemci ve sunucu tarafında kullanılan portlara iletişim portu (communication port) denir.

Kanal oluşturulduktan sonra mesajın boyutuna göre üç yöntemle haberleşilebilir:

- **Küçük mesajlar:** Hoca sınırı "yanlış hatırlamıyorsam 256 bayt" olarak verdi. Bu durumda iletişim portunun tampon alanı, mesajların yazıldığı ve okunduğu yer olarak kullanılır.
- **Daha büyük mesajlar:** Bir bölüm nesnesi (section object) tanımlanır. Section object, bu haberleşme kanalıyla ilişkili, belleğe eşlenmiş bir paylaşımlı bellek bölgesidir.
- **Section object'e sığmayacak kadar büyük veriler:** İstemci ve sunucunun adres alanına doğrudan yazmayı sağlayan farklı bir API kullanılır.

Bu üç yöntem, Bölüm 2.2'de sözü edilen "iki kez kopyalama" yükünün büyük veriler için nasıl atlatıldığını gösterir.

> **Ders sonrası not:** Hoca section object için "buradaki boyut sınırı biraz daha sınırlı" dedi ve sınırı hatırlayamadığını söyledi. Section object, 256 bayttan büyük mesajlar içindir; ondan da büyük veriler için doğrudan adres alanına okuma/yazma API'si kullanılır. Section object'in kesin bir boyut sınırı derste verilmedi.

14. Soketler

İstemci-sunucu haberleşmesinde kullanılan başka yapılar da vardır: soketler (sockets), uzak yordam çağrısı (RPC), borular (pipes) ve RPC'nin Java'daki bir başka sürümü olan uzak yöntem çağrısı (RMI, Remote Method Invocation). Hoca soketlerin ağ teknolojileri ve ileri ağ programlama derslerinde daha ayrıntılı konuşulduğunu, burada hızlıca üzerinden geçileceğini söyledi.

14.1 Soket Nedir?

Soket, haberleşme için tanımlanmış bir uç noktadır (endpoint). Farklı soket gerçeklemeleri vardır; slaytta soketler IP protokolü üzerinden tanımlanmış ve örneklenmişti.

TCP/IP yapısında süreçlerin haberleşmesi için iki adres bilgisi gerekir:

- **IP adresi:** Süreçler farklı cihazlarda olabilir. Bu yüzden cihazların adreslerine ihtiyaç vardır. Örneğin 161.25.19.8. Ama IP adresi yalnız hedefteki bilgisayarın kim olduğunu söyler.
- **Port numarası:** Haberleşme süreçler arasında gerçekleşir. Süreçlere ulaşmak için ek bir adresleme bilgisi gerekir. Bunu taşıma katmanı (transport layer) sağlar; port bilgisi burada devreye girer.

Soket, haberleşmenin yapılabilmesi için iki uçtaki adres bilgilerinin tutulduğu yapıdır: istemcinin IP adresi ve port numarası, sunucunun IP adresi ve sunucudaki sürece ait port numarası.

Bu dörtlü bir araya geldiğinde soketin tuttuğu bilgiler görülür. Çoğu zaman sokette üst tarafta kullanılan protokolün bilgisi de bulunur.

14.2 Portlar

Hoca TCP/IP'deki portların ilk 1024 tanesinin iyi bilinen (well-known) portlar olarak adlandırıldığını ve IANA tarafından standart hizmetlere ayrıldığını söyledi. Örnekler:

- 21: FTP
- 22: SSH
- 53: DNS
- 80: HTTP
- 443: HTTPS

Bunların üzerindeki portlar kendi ihtiyaçlarınız için ya da istemci tarafının kullanması içindir.

> **Ders sonrası not:** Hoca "1024 yani 2 üzeri 16 tane portumuz var" dedi. Doğrusu (Python ile 2^{16} hesaplandı): TCP ve UDP'de port numarası 16 bittir, toplam $2^{16} = 65536$ port vardır (0-65535). 1024 ise iyi bilinen portların sayısıdır (0-1023).

İstemci sunucuya ulaşmak ister. Bunun için hem sunucunun IP adresine hem de o makinede haberleşmek istediği sürecin adres bilgisine, yani port numarasına ihtiyaç vardır. Hocanın örneği: HTTP üzerinden bir istek gönderdiğinizde Google'a ait IP adresi bulunur ve o adresteki 80 numaralı portu dinleyen süreçle sizin tarayıcı (browser) süreciniz konuşur. Tarayıcı sürecinin port numarasının hiçbir önemi yoktur; isteği sistemde boş olan, kendi ürettiği herhangi bir port numarası üzerinden gönderebilir. Önemli olan, karşı tarafa her zaman erişebilmek için sunucunun sabit bir port numarasında olmasıdır.

14.3 Java ile Tarih Sunucusu

Slayttaki soket örneği Java ile verilmişti. Basit bir tarih sunucusudur (date server). Hoca bunun NTP sunucusuyla karıştırılmaması gerektiğini, yalnız tarihi söylediğini belirtti.

Sunucu 6013 numaralı portu dinleyen bir soket oluşturur. Sonra sonsuz bir `while` döngüsü içinde gelen istekleri kabul eder. Her istek için sistemin tarihini alır, soket üzerinden gönderir ve bağlantıyı kapatır. Sonra yeniden döngünün başına döner.

Hoca bu sunucunun çok süreçli (multi-process) ya da çok iş parçacıklı (multi-threaded) bir yapı olmadığına dikkat çekti. Gelen her isteği kabul edip işler ve yeni bir isteği bekler.

İstemci kısmı slaytta yoktu. Hoca istemcinin ne yapması gerektiğini sözlü anlattı: bir soket yaratmalı, soketi yaratırken hedef IP adresini ve o IP'de bağlanacağı sürecin port numarasını vermeli. Örneğin `new Socket("127.0.0.1", 6013)` gibi. Sonra bu soket üzerinden isteklerini gönderir. Bu örnekte protokol, istemcinin bir şey göndermesi yerine yalnız bağlantıyı başlatması şeklinde bırakılmıştır. Hocaya göre bu çok mantıklı olmamıştır: sistem gelen her bağlantıya bir cevap üretir; istemci bağlanır bağlanmaz tarihi alır.

15. Uzak Yordam Çağrısı (RPC)

Hoca RPC'nin doğrudan işletim sisteminin kapsamında olmadığını, bu yüzden çok ayrıntıya girmeyeceğini söyledi. Derdimiz şudur: yerelde bir yordamı (procedure) parametreleriyle çağırırız, çağrı yapılır ve biter; her şey çok temizdir. Aynı şeyi dağıtık bir ortamda da yapmak isteriz. Bunun için ortaya atılan yapı uzak yordam çağrısıdır (RPC, Remote Procedure Call).

Bu yapıda isteği yapan taraf istemci, isteği alıp karşılayan ve cevap üreten taraf sunucudur. İki taraf arasında haberleşirken iki tarafın birbirinden farklılıklarının ortadan kaldırılması gerekir. Yapmak istediğiniz şey bir fonksiyon çağrısıdır: hangi fonksiyonun çağrılacağını söyler ve ona parametre aktarırsınız. Ama iki sistem tamamen farklı yapıda çalışıyor olabilir:

- Biri little-endian, diğeri big-endian olabilir.
- Birindeki tamsayı (integer) boyutu diğerinden farklı olabilir.
- Veri tipi ya da çalışma mantığı bakımından yapılar çok farklı olabilir.

> **Ek bilgi (derste anlatılmadı):** Little-endian ve big-endian, büyük olasılıkla bilgisayar organizasyonu derslerinden bilindiği gibi, çok baytlı bir sayının bellekte hangi bayt sırasıyla saklandığını anlatır. Little-endian sistemde en düşük anlamlı bayt en düşük adreste, big-endian sistemde en yüksek anlamlı bayt en düşük adreste durur. Aynı dört baytı iki sistem farklı sayı olarak yorumlar. Bu yüzden ağ üzerinden gönderilen veride ortak bir gösterim kullanılır.

Bu farklılıkları ortadan kaldıracak, hem işleyişi hem parametreleri yönetecek yapılara ihtiyaç vardır. Bunlara koçan (stub) denir. Koçanın parametreleri ağdan gönderilebilecek biçime sokmasına sıralama (marshalling) denir. Koçanlar farklı sistemlerde farklı biçimlerde gerçeklenebilir. Örneğin Windows tarafında koçan kodu, MIDL (Microsoft Interface Definition Language) adlı bir tanım dilinde yazılmış belirtimden derlenerek oluşturulur.

16. Borular

Hoca, süreçler arası iletişim denince akla gelen en eski ve en ilkel yöntemin boru (pipe) olduğunu, bu yüzden üzerinde konuşmakta fayda olduğunu söyledi. Ayrıntıların fazlası uygulamada yapılacak.

16.1 Sıradan Borular

Sıradan boru (ordinary pipe), birbiriyle ilişkili süreçler arasında kullanılabilen bir yapıdır. İlişkiden kasıt, aralarında ebeveyn-çocuk ilişkisi olmasıdır.

Bir boru oluşturduğunuzda onu gerçekten bir boru gibi düşünebilirsiniz: borunun iki ucu da elinizdedir. Boru oluşturulunca size iki tanımlayıcı döner. Borunun bir ucuna yazıp öbür ucundan okuyabilirsiniz. Hoca bu tanımlayıcıları `fd[0]` ve `fd[1]` olarak adlandırdı.

Boru oluşturulduktan sonra `fork` çağrılırsa aynı boruya erişen ikinci bir süreç ortaya çıkar. Ebeveyn boruyu açmıştı; `fork` dediği anda bir de çocuk süreç vardır ve aynı uçlara erişebilir.

> **Ders sonrası not:** Hoca “ebeveyn borunun yazma ucuna sıfırdan, okuma ucuna birden erişiyor diyelim” dedi. Doğrusu tersidir: `pipe()` çağrısında `fd[0]` okuma ucu, `fd[1]` yazma ucudur. Bu, küçük bir C programıyla denendi: ebeveyn `fd[1]`’e yazdı, çocuk `fd[0]`’dan okudu.

Bu boru üzerinden haberleşme tek yönlü yapılır, çünkü biri yazarken diğeri de yazarsa ortalık karışır. Haberleşmenin ebeveynden çocuğa olması isteniyorsa iki taraf karşılıklı olarak kullanmadığı ucu kapatır. Ebeveyn yazan taraf olacaksa okuma ucunu kapatır; çocuk da yazma ucunu kapatır. Böylece borunun iki ucuna iki farklı süreç erişir hâle gelir.

Boru, mesaj geçirme bölümünde konuşulduğu gibi tek yönlü bir yapıdır. Ama farklı yönlerde çalışan iki boru kullanılarak çift yönlü bir yapı kurulabilir.

16.2 İsimli Borular (FIFO)

Sıradan borular yalnız ebeveyn-çocuk süreçler arasında kullanılabildiği için kullanımı sınırlıdır. Bunu geliştirmek için borular isimlendirilmiştir. Bunlara isimli boru (named pipe) denir, ama çoğu zaman FIFO adıyla görülür. Aralarında ebeveyn-çocuk ilişkisi olmayan süreçlerin de haberleşebilmesi için oluşturulmuş bir yapıdır.

İsimli boru `mkfifo` ile yaratılır. Oluşturulan FIFO, dosya sisteminde bir girdi (entry) olarak görünür. Dolayısıyla o adı bilen her süreç bu FIFO’ya erişebilir. Hoca ayrıntıların uygulama saatinde gösterileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Ders kitabına göre büyük olasılıkla UNIX’te bir FIFO’ya iki yönde de yazılıp okunabilir, ama aynı anda yalnız tek yönde veri akar (yarı çift yönlü, half-duplex). Bu yüzden iki yönlü eşzamanlı haberleşme gereken durumlarda genellikle iki FIFO kullanılır. Sıradan borulardan farklı olarak FIFO, onu kullanan süreçler sonlandıktan sonra da dosya sisteminde kalır; açıkça silinmesi gerekir.

Örnek Sorular

Soru 1 (derste sözlü, 00:04:32): `fork` çağrısından sonra ebeveyn ve çocuk süreçlerin süreç kontrol blokları teknik olarak aynıdır. Hangi iki bilginin farklı olması gerekir?

> **Çözüm (derste verildi):** Süreç kimliği (PID) ve ebeveyn süreç kimliği. Her sürecin ayrı bir kimliği olmalıdır. Her süreç başka bir süreç tarafından yaratıldığı için çocuğun ebeveyni onu yaratan süreçtir; ebeveyn-çocuk çiftinde bu değer de farklıdır. Karşılaştırma sistemdeki bütün süreçler için değil, ebeveyn-çocuk çifti içindir; aynı ebeveynin çocuklarının ebeveyn kimliği aynıdır.

Soru 2 (derste sözlü, 00:09:10): Mesaj kuyruğu çekirdek seviyesinde gerçeklenmişse, A sürecinin oluşturduğu bir mesajın B sürecine ulaşması için ne gerekir? Bu yöntem neden büyük veri blokları için uygun değildir?

> **Çözüm (derste verildi):** Her işlem için bir sistem çağrısı yapılması gerekir. Ayrıca mesaj en az iki kez kopyalanır: A'dan çekirdekteki kuyruğa, kuyruktan B'ye. Bu yüzden mesaj geçirme genelde küçük veriler için kullanılır.

Soru 3 (derste sözlü, 00:19:20): `in` ve `out` indeksleriyle gerçeklenen 10 gözlük dairesel tamponda tamponun boş mu dolu mu olduğu nasıl anlaşılır? Tampon en fazla kaç eleman tutar?

> **Çözüm (derste verildi):** Başlangıçta yazılacak göz de okunacak göz de 0'dır ve 0. gözde bir şey yoktur. Bu yüzden `in == out` tamponun boş olduğunu gösterir. Tampon `(in + 1) % BUFFER_SIZE == out` olduğunda doludur. Bu nedenle 10 gözlük tamponda en fazla `BUFFER_SIZE - 1 = 9` eleman tutulabilir. Sonuç C programıyla denendi: tampon 9 elemanda doldu.

Soru 4 (derste sözlü, 00:43:39): P1, P2 ve P3 aynı posta kutusunu paylaşıyor. P1 posta kutusuna bir mesaj gönderdi. Mesajı P2 mi alır, P3 mü?

> **Çözüm (derste verildi):** Bu bir tasarım kararıdır. Olası çözümler: bir posta kutusunun en fazla iki süreç tarafından paylaşılmasına izin vermek; herhangi bir anda yalnız bir sürecin `receive` yapmasına izin vermek; ya da alıcıyı sistemin (örneğin önceliğe göre) seçmesi ve mesajın kime verildiğini gönderene bildirmesi.

Soru 5 (derste sözlü, 01:05:55): Mach'ta posta kutusu doluyken mesaj işletim sistemi tarafından önbelleğe alınabiliyorsa, bir süreç dolu posta kutularına sürekli mesaj göndermeye çalışırsa ne olur?

> **Çözüm (derste verildi):** İşletim sistemine sürekli önbelleğe alma yükü gelir; bu uygulanabilir değildir. Bu yüzden önbelleğe alınabilecek mesaj sayısı bir ile sınırlandırılmıştır. Hoca sınırı posta kutusu başına verdi; ders kitabındaki kural büyük olasılıkla gönderen iş parçacığı başına bir bekleyen mesajdır (bkz. Bölüm 12.3).

Dersin Toparlaması

Bu derste süreçler arası iletişimin iki temel modeli ayrıntılı ele alındı. Paylaşımlı bellek, aynı makinedeki süreçler için kopyasız ve hızlı bir yoldur, ama senkronizasyonu sağlamak programcının sorumluluğundadır. Mesaj geçirme ise çekirdekte bir kuyruk ya da posta kutusu kullanır; en az iki kopyalama gerektirdiği için küçük veriler ve dağıtık sistemler için uygundur. Üretici-tüketici problemi ve dairesel tampon, iki modelde de ortaya çıkan "nereye yazıldı, nereden okunacak" sorusunu somutlaştırdı. Mesaj geçirme sisteminin tasarımında mesaj boyutu, doğrudan ya da dolaylı adresleme, bağlantının yönü, engelleme ve tamponlama kararları konuşuldu. Hem gönderme hem alma engelleyen olduğunda randevu oluştuğu, bunun üretici-tüketiciyi basitleştirse de iki süreci birbirinin hızına bağladığı görüldü.

Ardından gerçek sistemlerden örnekler incelendi: POSIX paylaşımlı belleğin `shm_open`, `ftruncate`, `mmap` ve `shm_unlink` adımları, her şeyin portlarla mesajlaştığı Mach mikroçekirdeği ve mesaj boyutuna göre üç farklı yol izleyen Windows ALPC. İstemci-sunucu haberleşmesi için soketler, RPC ve koçanlar, sıradan ve isimli borular konuşuldu. Bu dersle süreçler bölümü bitti. POSIX paylaşımlı bellek, mesaj kuyrukları ve borular uygulama saatlerinde Linux üzerinde gösterilecek. Sonraki derste iş parçacıkları (threads) bölümüne geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.